

How to Use AI

A Personal Experience

Dr. Vishvajeet Nagargoje

CNRS, LaBRI, Université de Bordeaux, France

August 2026

A personal, practical account of how I have come to use AI in everyday work.

The idea in one line: treat AI as a small team you *direct*, not a single assistant you *query* — one source of truth, one board of who is doing what, and many agents working in parallel. It comes in two tracks: a **simple path that needs no Git** for most people, and a **fuller Git workflow** for power users.

1. The core idea: a team, not a chatbot

The whole approach fits in one line: one **truth** (a single canonical store — a Git repository, or simply one shared folder), one shared **board** (who is doing what, and where), and many **hands** (agents working in parallel). Everything else is detail. I have come to treat agentic AI as a **development team I direct**, not a single assistant I query.

- Each **chat** — in Claude Code or Codex — is a team room. One chat can launch **several agents**, and each agent can spawn its own **sub-agents**. “Agent” means both the chat and the agents inside it.
- I can run **several chats at once** on the same project: more independent hands, more parallel progress.
- Agents perform essentially **any command-line task** — write and run code, move and rename files, run builds — and produce **any format** I ask for (L^AT_EX to PDF, Word, Excel, slides).
- Everything they do is anchored to **one shared truth** (Section 3).

2. Set up once

A little setup, done once, repays itself many times over.

1. Create a **project directory** — the single canonical folder for the work.
2. Give the agents **directory access**. Every tool opens the *same* folder, never a private copy.
3. Set up the **single source of truth** — a **Git host** (with API access), or just **one shared folder** if you would rather not use Git — and grant the agents **browser access**.
4. Fix one rule in everyone’s mind: **one canonical copy is the truth, and everyone syncs to it**.

3. The single source of truth — pick your track

Whatever else changes, one rule holds: there is a **single canonical copy** that everything syncs to. How you store it is the one place the setup differs by audience, so it comes in two tracks. Read the one that fits you — the rest of the guide is common to both.

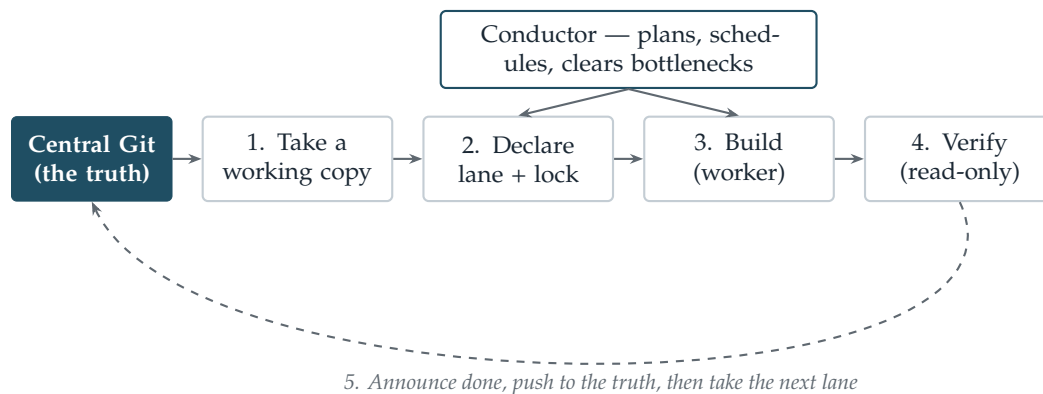
Track A — the simple path (no Git, for most people). The truth is a **single canonical folder** (on your machine, or a shared or synced drive). Each agent works on its own copy of the files it is touching; the board (Section 5) keeps two agents off the same file; the read-only verifier checks a change before it is folded back into the canonical folder; and you keep **dated backups** in place of a version history.

Track B — the power-user path (with Git). The central repository is the truth. Every agent takes an **isolated working copy** — a branch or a worktree — works there, and offers it back through a **pull request**; main is **protected**, so nothing lands without review, and the full history is kept for free.

Either way, **start oriented**: before touching files, I have one agent read the history and the current state, so it inherits past decisions instead of re-deriving them.

4. The working loop

Every agent runs the same short loop. A conductor plans and unblocks above it; a read-only verifier checks the work before it returns to the truth.



5. The board: staying aware of each other

Agents must be **aware of every other agent** — who is working, on what, and where.

- Before writing, an agent **announces** the lane it is taking, the **files** it will touch, and takes a **lock** on that scope.
- When finished, it **announces completion, returns its work to the truth, and picks up the next lane**. It stands down on any scope that is not its own.
- I keep the board as **one small record per lane**, plus a short presence note and an expiring lock per agent — *not* a single shared file, which quickly becomes a conflict magnet.

6. Two roles: conductor and worker

Their capabilities overlap; the conductor is also a worker.

Role	What it does
Conductor	Plans, splits the work, schedules lanes, integrates results, and clears bottlenecks — and does real work itself when that shortens the total time. Keeps the project on track.
Worker	Executes one bounded lane and produces durable, finished output.

On a small job it is often best to give **one agent everything** and have the others stand down, to avoid coordination overhead.

7. Plan for parallelism, then verify everything

I plan for the **shortest total time**, not the largest number of agents: independent lanes run in parallel, and overlapping ones are serialised. Each lane gets a **soft or hard preference** for which agent takes it — for example, *Claude preferred* for planning, *Codex* for heavy coding.

I assume the AI can be wrong, and never trust a single run:

- **Cross-check across systems.** Plan in one, review in the other; Claude and Codex each catch the other's mistakes.
- **Send read-only verifiers** alongside the writers. A verifier inspects the evidence; it does not simply agree.
- **Repeat the build-then-verify loop.** For large or complex work, I run it many times.
- **Treat runs as samples.** Run the same task several times, alternating Claude and Codex, and take the best or the consensus result. Scale the number of runs to the risk.

8. Choose the agent, model, and effort

- **Play to strengths** (preferences, not fixed rules): Claude tends to be stronger at **planning**; Codex is **faster at writing and running code**.
- **Match effort to the task.** For technical work a **high** or **extra-high** setting usually pays off; I dial it down for routine work.
- **Mind the cost.** A top-tier model at high effort handles most technical work well. The newest models are more capable but **consume more tokens** — I keep an eye on usage.

9. Agents as researchers and crawlers

I point an agent at something to **learn its shape** before acting on it:

- A repository's **structure** and decision history, or **another agent's work** (read from Git).
- A website's **sitemap**, or any large external domain.

Browser crawling. With browser access, an agent can navigate by pointer and scroll, **take screenshots**, and even critique a design visually. For anything **behind a login**, I **sign in first**, then hand the session to the agent.

- *Finding the lowest price:* I have an agent crawl shopping sites and report the cheapest option for an item.

- *Portal tasks* (for example an HR portal): I sign in myself, then have the agent read my records or complete a form — work a plain API cannot reach.

A good researcher records **sources, dates, coverage, and uncertainty**, so later agents reuse the result instead of re-crawling.

10. Timers, monitors, and long runs

- I set a **one-off or recurring timer** of any length for a task — for example, “review the churn over the next six hours” — and use it to **check on another agent** or on external state.
- **Large projects need several runs** over a longer time. Each run should leave **resumable state** so the next one continues cleanly.

11. Talking to it: context is everything

- **Give as much context as you can.** The more it knows about you and the project, the better it works — more feedback, better results.
- **Do not over-structure.** Typos, jumbled words, and dictation are fine; volume and honesty of context beat polish.
- **Just talk.** I dictate through the browser or the operating system’s speech-to-text, or have the AI transcribe me, then pass it along. Talking often beats typing.
- **Keep everyone oriented** on the current state, and move in **small, incremental steps**.

12. Start from something, not from scratch

It is better — for the AI and for people — to **start from an existing project and improve it** than to build from zero. I would not write, say, a scheduler from scratch: I take an open-source scheduler, add my requirements, and re-skin it. Both the AI and I do far better on the second path.

13. Automate the repeats

Anything I do often graduates from ad-hoc instructions into a **reusable component**: a **skill** (a saved workflow), a **connector** (a safe interface to an external system), or a **skin** (a reusable presentation layer). I define each one’s inputs, outputs, permissions, and failure behaviour.

None of this is exotic. The shift that made the difference was one of stance: treating the AI as a team to be directed, kept honest by cross-checking and independent verification, and anchored at all times to a single source of truth.

Acknowledgment. Thanks to Apuro Nellika for scribing and helping prepare this document.